

Relatório

Análise de complexidade sobre estruturas de listas

Alunos: Dominic Cassidy, Gabriel Lacerda, Marcos Paulo

Repositório do projeto:

https://github.com/Pieisnolie/TRAB_1_TPA

Relato sobre o desenvolvimento da biblioteca e do programa de teste

Nós fizemos o trabalho praticamente todo em conjunto. Durante o desenvolvimento, nos reunimos em chamadas e usamos o compartilhamento de tela para acompanhar o código, trocar ideias e ajudar quando apareciam dúvidas ou erros. Não dividimos as tarefas de forma muito separada, porque todos foram participando das etapas e contribuindo.

Utilização de ferramentas de Inteligência Artificial

Durante o desenvolvimento do trabalho, foram utilizadas ferramentas de Inteligência Artificial como apoio para interpretar melhor o enunciado, organizar o que foi pedido pelo professor. A IA ajudou principalmente a transformar o roteiro do trabalho em etapas mais claras de desenvolvimento.

Também foi utilizada para revisar trechos do código, ajudar a encontrar erros durante os testes e auxiliar o grupo com o uso do "Comparator" para trabalhar com as listas ordenadas. Além disso, a IA serviu como apoio para organizar a parte de análise de complexidade e os testes de tempo do programa.

Ferramentas utilizadas: Chat GPT, IntelliJ IDEA, Git, Google docs

Análise Matemática de complexidade dos Algoritmos da biblioteca

Método adicionar

```
19  public boolean adicionar(T novoValor) {
20
21     No<T> novo = new No<>(novoValor);
22
23     if (prim == null) {
24         prim = novo;
25         return true;
26     }
27
28     if (!ehOrdenada) {
29         novo.setProx(prim);
30         prim = novo;
31         return true;
32     }
33
34     if (comparador.compare(novoValor, prim.getValor()) <= 0) {
35         novo.setProx(prim);
36         prim = novo;
37         return true;
38     }
39
40     No<T> atual = prim;
41
42     while (atual.getProx() != null
43         && comparador.compare(
44             atual.getProx().getValor(),
45             novoValor) < 0) {
46
47         atual = atual.getProx();
48     }
49
50     novo.setProx(atual.getProx());
51     atual.setProx(novo);
52
53     return true;
54 }
```

O primeiro if(linha 23) é o melhor caso, onde a lista está vazia e o elemento é adicionado no primeiro espaço, o segundo if(linha 28) é o caso de uma lista não ordenada, onde o elemento é inserido sempre no início, neste caso a complexidade no pior caso é $O(1)$, já que sempre será adicionado no primeiro espaço da lista. O terceiro if(linha 34) é para uma lista ordenada onde o elemento é inserido antes do primeiro, o while da linha 42 é quando o elemento é inserido no meio ou no fim da lista, sendo o pior caso a inserção no fim, quando tem que se percorrer a lista inteira, nesse caso a complexidade é $O(n)$ para lista ordenada, já que não temos if/while se sobrepondo.

Método pesquisar

```
57  public T pesquisar(T valor) {
58
59      No<T> atual = prim;
60
61      while (atual != null) {
62
63          int resultadoComparacao =
64              comparador.compare(atual.getValor(), valor);
65
66          if (resultadoComparacao == 0) {
67              return atual.getValor();
68          }
69
70          if (ehOrdenada && resultadoComparacao > 0) {
71              return null;
72          }
73
74          atual = atual.getProx();
75      }
76
77      return null;
78  }
```

Para o melhor caso de pesquisa, ou seja, quando o elemento pesquisado é o primeiro da lista, o programa encontra o elemento logo no início. No pior caso, o elemento está no final da lista, fazendo com que seja necessário percorrer todos os nós. Em uma lista não ordenada, caso o elemento não exista, também será necessário percorrer toda a lista. Já na lista ordenada, a busca pode terminar antes quando o programa percebe que o elemento não poderá mais ser encontrado. Mesmo assim, no pior caso, a complexidade da pesquisa é $O(n)$ tanto para listas ordenadas quanto não ordenadas.

Método remover

```
81  public boolean remover(T valor) {
82
83     if (prim == null) {
84         return false;
85     }
86
87     int comparacaoPrimeiro =
88         comparador.compare(prim.getValor(), valor);
89
90     if (comparacaoPrimeiro == 0) {
91         prim = prim.getProx();
92         return true;
93     }
94
95     if (ehOrdenada && comparacaoPrimeiro > 0) {
96         return false;
97     }
98
99     No<T> anterior = prim;
100     No<T> atual = prim.getProx();
101
102     while (atual != null) {
103
104         int resultadoComparacao =
105             comparador.compare(atual.getValor(), valor);
106
107         if (resultadoComparacao == 0) {
108             anterior.setProx(atual.getProx());
109             return true;
110         }
111
112         if (ehOrdenada && resultadoComparacao > 0) {
113             return false;
114         }
115
116         anterior = atual;
117         atual = atual.getProx();
118     }
119
120     return false;
121 }
```

No primeiro if (linha 83) é verificado se a lista está vazia. No segundo if (linha 90) é verificado se o elemento a ser removido é o primeiro nó. Já o terceiro if (linha 95) permite encerrar a busca mais cedo em uma lista ordenada quando o elemento não pode mais ser encontrado. O while (linha 102) percorre a lista usando o elemento atual e o anterior para permitir a remoção de um elemento que esteja no meio ou no final. O pior caso ocorre quando o elemento a ser removido está no final da lista, sendo necessário percorrer todos os nós. Nesse caso, tanto para listas ordenadas quanto não ordenadas, a complexidade é $O(n)$.

Método quantidadeNos

```
124  public int quantidadeNos() {  
125  
126     int quantidade = 0;  
127     No<T> atual = prim;  
128  
129     while (atual != null) {  
130         quantidade++;  
131         atual = atual.getProx();  
132     }  
133  
134     return quantidade;  
135 }
```

Esse método conta a quantidade de nós na lista, como ele sempre tem que percorrer a lista toda, a complexidade dele é sempre $O(n)$.

Análise Empírica de complexidade de Algoritmos da biblioteca

Quantidade de contatos	Tipo de lista	Montagem (ms)	Busca por telefone (ms)	Busca por nome (ms)	Remoção (ms)
25000	Não ordenada	28,9951	2,5468	2,4121	1,9682
25000	Ordenada	6.456,4597	2,9280	2,4510	2,0944
50.000	Não ordenada	67,1080	5,4951	2,5436	4,0543
50.000	Ordenada	33.569,6673	4,0160	2,0573	4,2598
75.000	Não ordenada	95,5359	5,9717	2,1371	5,7433
75.000	Ordenada	72.717,4664	5,9204	1,7158	7,9216
100.000	Não ordenada	127,5194	6,9442	3,7597	8,1935
100.000	Ordenada	137.549,9531	6,6712	2,2508	6,2533

Interpretação dos resultados

Os resultados dos testes ficaram, de forma geral, de acordo com o que foi visto na análise matemática. A maior diferença aparece no tempo de montagem das listas.

Na lista não ordenada, o método adicionar possui complexidade $O(1)$, pois o novo elemento é colocado diretamente no início da lista. Por isso, mesmo aumentando a quantidade de contatos, o tempo de montagem continuou relativamente baixo.

Já na lista ordenada, no pior caso, o método adicionar possui complexidade $O(n)$, pois pode ser necessário percorrer a lista até encontrar a posição correta. Nos arquivos utilizados nos testes, os contatos já estavam em ordem crescente, fazendo com que os novos elementos fossem inseridos no final da lista. Assim, esse caso ficou próximo do pior caso analisado anteriormente. Como o método adicionar é executado para cada contato do arquivo, o tempo total de montagem aumentou muito conforme o tamanho da entrada aumentou.

Nas pesquisas, o resultado também ficou de acordo com a análise matemática. Como foi pesquisado o último elemento da lista, foi necessário percorrer praticamente toda a estrutura, representando o pior caso de $O(n)$.

Na remoção aconteceu o mesmo. Como o elemento removido era o último da lista, foi necessário percorrer os elementos até encontrá-lo, ficando também de acordo com a complexidade $O(n)$ encontrada na análise matemática.

Os tempos de pesquisa e remoção não aumentaram de forma exata em todos os testes. Em alguns casos, uma entrada maior apresentou um tempo um pouco menor. Essas diferenças podem acontecer porque os tempos são muito pequenos e podem variar entre as execuções. Mesmo assim, o funcionamento do algoritmo continua de acordo com a análise feita, pois no pior caso a quantidade de elementos percorridos aumenta junto com o tamanho da lista.